

Taller de bucles (loops) en JavaScript

Recorriendo la app de Nequi con for, while, do...while, break, continue y bucles anidados

Programming Foundations · Generation Colombia

Antes de empezar

Toda la práctica de hoy vive dentro de una misma aplicación: **Nequi**. Vas a resolver problemas reales que la app tiene que hacer todos los días: sumar los movimientos del mes, validar el PIN de un usuario, mostrar el menú, buscar una transacción y revisar varias cuentas a la vez.

Cada ejercicio te da la **situación de negocio**, los datos con los que trabajas y una guía paso a paso. **No te damos el código resuelto**: el objetivo es que tú lo escribas. Te sugerimos nombres de variables para que hablemos el mismo idioma.

Ejercicio 1 · Los movimientos del mes

La situación

A fin de mes, Nequi necesita mostrarte cuánto dinero se movió en tu cuenta. Tenemos una lista con los movimientos del mes y queremos **sumar todos** para saber el total. Además, queremos **contar cuántos de esos movimientos fueron retiros**.

Para pensar

Si tienes 30 movimientos, ¿escribirías 30 sumas a mano, una por una? ¿Qué pasa si mañana son 500 movimientos? ¿Hay una forma de decirle al programa "haz esto por cada movimiento" sin repetir el código?

Con qué trabajas

Parte de una lista de movimientos ya creada. Cada movimiento tiene un valor (positivo si entra dinero, negativo si sale) y un tipo. Usa nombres como `movimientos` para la lista, `total` para ir acumulando la suma y `cantidadRetiros` para el conteo.

Paso a paso

1. Crea el archivo `ejercicio1.js` en tu carpeta del taller.
2. Declara la lista `movimientos` con al menos 6 valores (mezcla entradas y salidas de dinero).
3. Antes del bucle, crea una variable `total` en 0 y otra `cantidadRetiros` en 0. Piensa por qué deben empezar en 0.
4. Usa un bucle `for` que recorra la lista desde la primera posición hasta la última.
5. Dentro del bucle, en cada vuelta suma el valor actual a `total`.
6. Sigue dentro del bucle: usa un `if` para preguntar si el movimiento es un retiro (por ejemplo, si su valor es negativo). Si lo es, súmalo 1 a `cantidadRetiros`.
7. Después del bucle (¡fuera de él!), muestra en consola el `total` y la `cantidadRetiros`.
8. Ejecuta con `node ejercicio1.js` en la terminal y verifica los números a mano.

Para pensar

¿Qué pasa si pones el `console.log` del total *dentro* del bucle en vez de afuera? ¿Cuántas veces se imprime? ¿Cuál es el valor "bueno", el último o los del camino?

Ejercicio 2 · Validar el PIN

La situación

Para entrar a Nequi debes escribir tu PIN. La app te deja **volver a intentar** si te equivocas. Aquí hay algo distinto al ejercicio anterior: **no sabemos cuántas veces lo va a intentar el usuario**. Puede acertar de una, o fallar cinco veces.

Para pensar

Un `for` es ideal cuando sabes cuántas vueltas dar. Pero aquí no lo sabes: depende de cuándo acierte el usuario. ¿Sirve el `for` para esto? ¿Qué tipo de bucle repetiría "mientras el PIN sea incorrecto"?

Nuevo: leer lo que el usuario escribe

Hasta ahora tus datos estaban escritos en el código. Ahora necesitas que el **usuario escriba** el PIN por teclado. Como estamos en Node.js (no en el navegador), usamos una herramienta llamada `prompt-sync`. Esta es la *única* parte del taller donde te damos comandos exactos, porque es instalación de una herramienta nueva.

1) En la terminal, dentro de tu carpeta del taller, instala la herramienta:

```
npm install prompt-sync
```

Esto descarga `prompt-sync` y crea la carpeta `node_modules`. Solo se hace una vez por carpeta.

2) Al inicio de tu archivo, activa la herramienta con estas dos líneas:

```
const prompt = require('prompt-sync')();
```

A partir de ahí, cuando escribas `prompt('Escribe tu PIN: ')`, el programa se detiene, muestra ese texto y espera a que el usuario escriba y presione Enter. Lo que escriba queda guardado en una variable.

Ojo con un detalle: `prompt` te devuelve **texto**. Si vas a comparar con un número, ten presente que `"1234"` (texto) no es lo mismo que `1234` (número). Decide con qué vas a comparar.

Con qué trabajas

Guarda el PIN correcto en una variable como `pinCorrecto`. Guarda lo que escribe el usuario en `intento`. Puedes llevar la cuenta de intentos en `numeroIntentos` si quieres.

Paso a paso

9. Crea el archivo `ejercicio2.js` e instala `prompt-sync` como se explicó arriba.
10. Define el `pinCorrecto` (invéntate uno).
11. Pide el `intento` una primera vez con `prompt`.
12. Usa un bucle `while` que se repita *mientras* el intento sea distinto al PIN correcto.

13. Dentro del bucle: avisa que el PIN es incorrecto y **vuelve a pedir** el intento (si no lo pides de nuevo, el bucle nunca cambia... piensa qué pasaría).
14. Cuando el usuario acierte, el bucle termina solo. Fuera del bucle, dale la bienvenida a la app.
15. Ejecuta con `node ejercicio2.js` y pruébalo fallando a propósito varias veces.

Para pensar

¿Qué pasa si *olvidas* volver a pedir el intento dentro del bucle? El PIN incorrecto nunca cambia... ¿el programa se detiene alguna vez? A eso se le llama un **bucle infinito**. ¿Cómo lo evitas siempre?

Ejercicio 3 · El menú de la app

La situación

Ya entraste a Nequi. Ahora la app te muestra un **menú**: 1) Ver saldo, 2) Enviar dinero, 3) Recargar, 4) Salir. El menú debe mostrarse, esperar tu elección, y **volver a mostrarse** después de cada acción, hasta que elijas Salir.

Para pensar

El menú tiene que aparecer **al menos una vez**, sí o sí, antes de que el usuario decida algo. Con un `while` normal, primero se revisa la condición y luego se entra. ¿Existe un bucle que *primero actúe y después pregunte* si repite?

Con qué trabajas

Guarda la elección del usuario en `opcion` (la pides con `prompt`). El bucle se repite hasta que `opcion` sea la de Salir.

Paso a paso

16. Crea `ejercicio3.js`. Reutiliza la activación de `prompt-sync` del ejercicio anterior.
17. Usa un bucle `do...while`: el bloque `do` se ejecuta primero, y la condición `while` se revisa al final.
18. Dentro del `do`: muestra las 4 opciones del menú y pide la `opcion`.
19. Usa `if` (o varios) para reaccionar a cada opción con un mensaje distinto. Aún no hace falta que "funcione" de verdad: basta con un mensaje por opción.
20. La condición del `while` al final debe mantener el menú vivo *mientras* la opción **no** sea Salir.
21. Ejecuta con `node ejercicio3.js` y recorre el menú varias veces antes de salir.

Para pensar

¿En qué se diferencia este `do...while` del `while` del ejercicio 2? Si la primera vez el usuario ya escribiera "Salir", ¿el menú alcanzaría a mostrarse al menos una vez? ¿Y con un `while` normal?

Ejercicio 4 · Buscar y filtrar movimientos

La situación

Nequi necesita dos cosas nuevas sobre la lista de movimientos: (a) **encontrar el primer pago** a un comercio y avisar en cuál posición está, y (b) al recorrer la lista, **ignorar los movimientos de \$0** (movimientos vacíos que no interesan).

Para pensar

Si estás recorriendo la lista buscando **el primer pago** y ya lo encontraste en la posición 2... ¿tiene sentido seguir revisando las otras 40 posiciones? ¿Cómo le dices al bucle "ya, *detente*"? Y por otro lado, ¿cómo le dices "este no me sirve, *sáltatelo* y sigue con el próximo"?

Las dos herramientas nuevas

- **break** → **rompe** el bucle por completo. En cuanto se ejecuta, el bucle termina, aunque queden vueltas.
- **continue** → **salta** el resto de *esta* vuelta y pasa directo a la siguiente. No rompe el bucle, solo se salta lo que falta.

Analogía: imagina que revisas una fila de personas buscando a alguien. **break** es encontrarlo y **irte de la fila**. **continue** es ver que la persona actual no es y **pasar a la siguiente** sin irte. **Dónde se rompe la analogía:** en la fila real tú decides con la vista; en el código la decisión la toma un **if** con una condición exacta.

Con qué trabajas

Reutiliza una lista `movimientos`. Para la búsqueda, define qué es "un pago a comercio" (por ejemplo, un tipo específico). Puedes guardar la posición encontrada en `posicionEncontrada`.

Paso a paso

22. Crea `ejercicio4.js` con una lista de movimientos variada (incluye algunos en 0 y algún pago a comercio).
23. Recorre la lista con un `for`.
24. Parte A — filtrar: al inicio de cada vuelta, usa un `if` que detecte si el movimiento es 0. Si lo es, usa `continue` para saltarte esa vuelta.
25. Parte B — buscar: con otro `if`, detecta si el movimiento es el pago a comercio que buscas. Si lo es, muestra su posición y usa `break` para detener el recorrido.
26. Ejecuta con `node ejercicio4.js`. Cambia el orden de la lista y observa cómo cambia la posición encontrada.

Para pensar

Después del **break**, ¿se revisan los movimientos que venían después? Si *quitaras* el **break**, ¿encontrarías el **primer pago** o el **último**? ¿Por qué cambia el resultado?

Ejercicio 5 · Varias cuentas a la vez

La situación

Nequi no tiene un solo usuario: tiene **muchos**, y cada usuario tiene **su propia lista de movimientos**. La empresa quiere el total gastado **por cada usuario**. O sea: por cada usuario, recorrer todos sus movimientos.

Para pensar

Ya sabes recorrer **una** lista de movimientos (ejercicio 1). Ahora tienes **una lista de usuarios**, y *cada usuario* tiene su lista de movimientos. ¿Cuántos recorridos necesitas: uno solo, o uno *dentro* de otro?

La idea: un bucle dentro de otro

Un **bucle anidado** es un bucle dentro de otro. El **de afuera** recorre los usuarios; por *cada* usuario, el **de adentro** recorre sus movimientos. El de adentro se completa *entero* antes de que el de afuera pase al siguiente usuario.

Con qué trabajas

Una lista **usuarios**, donde cada usuario tiene un nombre y su lista de **movimientos**. Usa **totalUsuario** para acumular el gasto de *cada* uno.

Paso a paso

27. Crea **ejercicio5.js** con al menos 3 usuarios, cada uno con su propia lista de movimientos.
28. Bucle de afuera: recorre **usuarios** con un **for**.
29. Dentro, por cada usuario, crea **totalUsuario** en 0. Piensa: ¿por qué debe reiniciarse a 0 en cada usuario y no una sola vez al principio?
30. Bucle de adentro: recorre los **movimientos** de ese usuario y ve sumándolos a **totalUsuario**.
31. Al terminar el bucle de adentro (pero aún dentro del de afuera), muestra el nombre del usuario y su total.
32. Ejecuta con **node ejercicio5.js** y verifica que cada usuario tenga su propio total.

Para pensar

¿Qué pasaría si **totalUsuario** lo creas **una sola vez** antes del bucle de afuera, en vez de reiniciarlo por usuario? ¿Los totales seguirían siendo correctos? Además: en los ejercicios usaste **for** para recorrer listas... ¿recuerdas algún método de arrays que también recorra? ¿Podrías lograr lo mismo con él?

Para cerrar

Recorriste toda la app de Nequi usando el bucle correcto para cada situación: **for** cuando sabías cuántas vueltas dar, **while** cuando dependía de una condición, **do...while** cuando algo debía pasar al menos una vez, y **break/continue** para controlar el recorrido por dentro. Y con los anidados, resolviste "algo por cada cosa, dentro de otra cosa".